

LAMBDA, CURRYING, AND ENVIRONMENTS Solutions

COMPUTER SCIENCE MENTORS 61A

September 14, 2026 - September 18, 2026

1 Expressions and Truth Values

```
1. def quick_maths(man, free):  
    print(man)  
    if man % 2 == 0 and free:  
        return quick_maths(man + 2, not free)  
    else:  
        man -= 1  
        if man == 3:  
            print(man)  
        else:  
            return man  
big_shaq = 2
```

Expression	Interactive Output
<code>print(quick_maths(big_shaq, True))</code>	

Expression	Interactive Output
<code>print(quick_maths(big_shaq, True))</code>	2 4 3 None

2 Lambda Expressions and Function Forms

1. What do lambda expressions do?

Lambda expressions create functions in the environment. When you evaluate one, you get out a function (feel free to call them a lambda function) as a value.

2. Rewrite each function in the requested equivalent form. Preserve the function's parameters and behavior; only change whether it is written with **def** or **lambda**. For your reference, a **lambda** assignment assigns a lambda function to a variable.

- (a) Rewrite the following **def** as a lambda assignment.

```
def square(x):  
    return x * x
```

```
square = lambda x: x * x
```

- (b) Rewrite the following lambda assignment using **def**.

```
subtract = lambda x, y: x - y
```

```
def subtract(x, y):  
    return x - y
```

- (c) Rewrite this zero-argument function as a lambda assignment.

```
def greeting():  
    return 'Hello, world!'
```

```
greeting = lambda: 'Hello, world!'
```

3 Currying and Zero-Argument Functions

1. Fill in the blanks (*without using any numbers in the first blank*) such that the entire expression evaluates to 9.

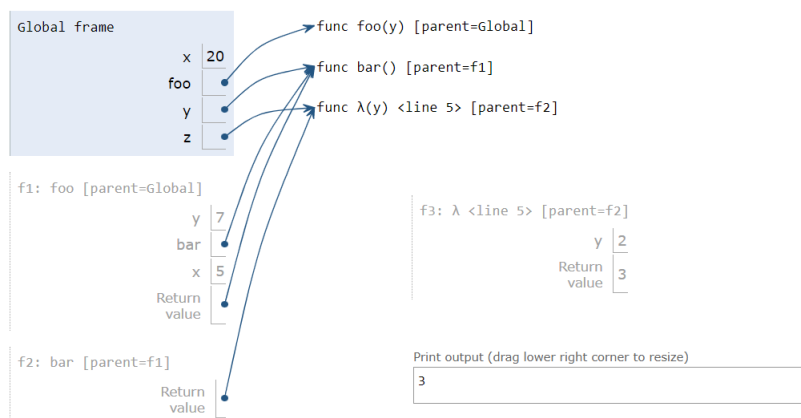
```
(lambda x: lambda y: lambda: y(x)) (3) (lambda z: z*z) ()
```

4 Environment Diagrams with Lambdas

1. Give the environment diagram and console output that result from running the following code.

```
x = 20
def foo(y):
    x = 5
    def bar():
        return lambda y: x - y
    return bar
```

```
y = foo(7)
z = y()
print(z(2))
```

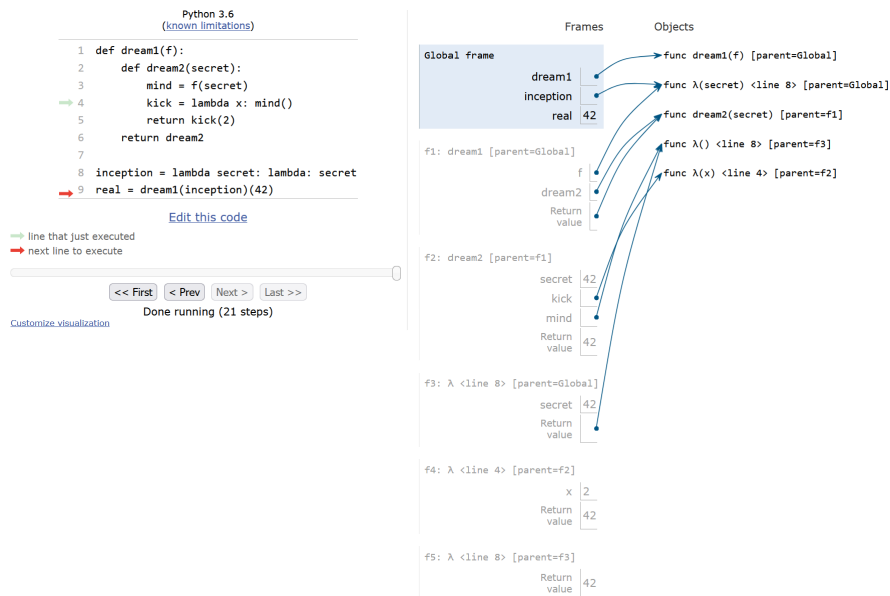


<https://tinyurl.com/yxfcvxxa>

2. Draw the environment diagram that results from running the code below.

```
def dream1(f):  
    def dream2(secret):  
        mind = f(secret)  
        kick = lambda x: mind()  
        return kick(2)  
    return dream2  
  
inception = lambda secret: lambda: secret  
real = dream1(inception)(42)
```

Output: 42



<https://imgur.com/a/ZKwZzdy>

5 Basic Function Implementation

1. Write a higher-order function that passes the following doctests.

Challenge: Write the function body in one line.

```
def mystery(f, x):
    """
    >>> from operator import add, mul
    >>> a = mystery(add, 3)
    >>> a(4) # add(3, 4)
    7
    >>> a(12)
    15
    >>> b = mystery(mul, 5)
    >>> b(7) # mul(5, 7)
    35
    >>> b(1)
    5
    >>> c = mystery(lambda x, y: x * x + y, 4)
    >>> c(5)
    21
    >>> c(7)
    23
    """

    def helper(y):
        return f(x, y)
    return helper
```

Challenge solution:

```
return lambda y : f(x, y)
```

2. Implement `make_skipper`, which takes in a number `n` and outputs a function. When this function takes in a number `x`, it prints out all the numbers between 0 and `x`, skipping every `n`th number.

```
def make_skipper(n):
    """
    >>> a = make_skipper(2)
    >>> a(5)
    1
    3
    5
    """

    def skipper(x):
        for i in range(x + 1):
            if i % n != 0:
                print(i)
    return skipper
```

